



[BIT703 Web Technologies](#) / [Contents](#) / Topic 6: Client-side web application development – CSS / Understanding CSS

Flexbox 60m

One of the more powerful options to use to create layouts is called flexbox.

Flexbox is short for Flexible Box Module and can be described as:

‘a one-dimensional layout method for laying out items in rows or columns. Items flex to fill additional space and shrink to fit into smaller spaces.’ (MDN, 2022, para. 1)^{fnref:a48e60}

Flexbox is a modern tool that helps us design web page layouts, so we don’t need to use the `<table>...</table>` tags, which were used in the past. While `<table>` tags can create layouts, they’re not responsive, meaning they don’t adjust well to different screen sizes.

Later in this topic, we’ll explore ‘grid’, which is an even better option for creating page layouts.

To use flexbox, you first need to create a flex container. This is a box that holds all the other boxes, and they’ll follow the rules you set for the container. To use flexbox in CSS, you start by creating a `<div>` and setting its display property to flex, like this:



[Skip to main content](#)



```
1 .one {  
2   display: flex;  
3 }
```



All modern browsers support flexbox and do not need prefixes.

In example 6.12, we have taken a set number of cells and displayed them as a grid.

Can you spot the crucial code for the flexbox?



[Skip to main content](#)

HTML

CSS

Result

EDIT ON

LIVE

```
.grid {
  display: flex;
  flex-direction: column;
  overflow: auto;
  margin: 0 auto 30px;
}

.grid .row {
  display: flex;
  flex-grow: 1;
  margin-bottom: 1rem;
}

.grid .row:last-of-type {
  margin-bottom: 0;
}

.grid .cell {
  flex-grow: 1;
}

.grid.columns {
  flex-direction: row;
  flex-wrap: wrap;
}

.grid.columns .cell {
  margin-bottom: 1rem;
}

h3{
  text-align: center;
}

.container {
  border: 1px solid #ccc;
```

Resources1x0.5x0.25xRerun

Example 6.12 Flexbox



☒ What is the code?

Non-Assessed

Status: Done
Skip to main content

Reset ↺

Based on the CodePen example above, what section of the code is the crucial part needed to create this flexbox?

Submit the code in the box below.

```
.grid {  
  display: flex;  
  flex-direction: column;  
  overflow: auto;  
  margin: 0 auto 30px;  
}
```

11 / 500 Word Limit

Here is the crucial code:

```
.grid {  
  display: flex;  
  flex-direction: column;  
  overflow: auto;  
}  
.grid .row {  
  display: flex;  
  flex-grow: 1;  
}  
.grid .cell {  
  flex-grow: 1;  
}
```

Previous attempts ▼



[Skip to main content](#)

Dig deeper

If you want to further explore the options available in flexbox with some visuals before continuing, read:

- *CSS Flexbox Layout Guide* (Coyier, 2024),^{fnref:e2a25a} which explains more ways that flexbox can be used to create layouts.

Understanding the code

So, what does the flexbox code displayed in the CodePen example above mean? What does the code do?

Code	Meaning of code	What code does
.grid { display: flex;	.grid is set to display flex	All its direct children (.row) can use flexbox properties.
.grid { display: flex flex-direction: column;	.grid is set in the column direction	The .row elements will display up-to-down, not left-to-right.
.grid .row { display: flex;	.row is set to display flex	All its direct children (.cell) can use flexbox properties.
.grid .row { display: flex; flex-grow: 1; margin-bottom: 1rem;	.row is not set in the column direction	The .cell elements will default to left-to-right display.
(*no flex-direction included in this code*)		
.grid .row { display: flex; flex-grow: 1;	.row has a flex-grow value of 1	All elements inside will have an equal share of the parent space.

Skip to main content

How can flexbox help with layout?

Now let's look at some of the ways flexbox can help us easily lay out items. A traditional navigation menu with each page listed using the element will get you the layout shown in the CodePen example below.

Can you see that the navigation menu layout is displayed vertically?



HTML

CSS

Result

EDIT ON

LIVE

```
.container {  
  border: 1px solid #ccc;  
  margin: 15px auto;  
  max-width: 50rem;  
  padding: 0 30px;  
}  
.cell {  
  background-color: rgba(0, 0, 0,  
0.05);  
}  
.cell {  
  border: 1px solid rgba(0, 0, 0,  
0.2);  
  border-right: none;  
  padding: 1em;  
  text-align: center;  
}  
.cell:last-of-type {  
  border-right: 1px solid rgba(0, 0,  
0, 0.2);  
}
```

Skip to main content

Resources

1x 0.5x 0.25x

Rerun

Example 6.13 Flexbox



Now we can use flexbox to control the layout of the navigation menu. When you add `display:flex` to the `nav > ul` element, it defaults to the following settings:

- **flex-direction: row;** – puts all child `<ol>` elements in a single row (much like `display:inline;`)
- **justify-content: flex-start;** – puts all child `<ol>` elements at the start.

By adding `display:flex`, we control the layout of the direct descendants – the first-level children. In the CodePen example below, you can see what adding `display:flex` to the CSS does to the navigation menu.

Can you see the difference between **Example 6.13** and 6.14 from just adding `display:flex` to the `<ol>` element?



[Skip to main content](#)

HTML

CSS

Result

EDIT ON

LIVE

```

ul{
  display: flex
}
.container {
  border: 1px solid #ccc;
  margin: 15px auto;
  max-width: 50rem;
  padding: 0 30px;
}
.cell {
  background-color: rgba(0, 0, 0,
0.05);
}
.cell {
  border: 1px solid rgba(0, 0, 0,
0.2);
  border-right: none;
  padding: 1em;
  text-align: center;
}
.cell:last-of-type {
  border-right: 1px solid rgba(0, 0,
0, 0.2);
}

```

Resources
1x 0.5x 0.25x
Rerun

Example 6.14 Flexbox 2



Now we can apply rules to the flex container to determine what will happen to the elements inside it. Some of the most common layout options are listed below.

Layout option	Definition	Examples
Skip to main content		

flex-direction	Determines whether the flex items are displayed in a row or a column. You can also reverse the stack to change the order.	flex-direction: row; flex-direction: column;
flex-wrap	Determines whether the flex items should wrap to a new row or column if they don't fit across the dimensions of the flex container.	flex-wrap: no-wrap; flex-wrap: wrap; flex-wrap: wrap-reverse;
justify-content	Justify-content is used to position items evenly within a container.	justify-content: center; justify-content: space-around; justify-content: space-between; justify-content: flex-start; justify-content: flex-end;

Justify-content

As you can see in the table above, there are a few values you can use with justify-content. These are:

Skip to main content

- center – aligns the items in the centre of the container

- space-around – aligns the items with equal space on either side of each item
- space-between – aligns the items with equal space between the items, but not at the ends of the container
- flex-start – places the items at the start of the container
- flex-end – places the items at the end of the container.

Let's now try to apply these justification styles in the activity below.



[Skip to main content](#)

Example 6.15 Task: Flexbox justification



[Skip to main content](#)

☒ Flexbox justification

Non-Assessed

Status: Done

Reset ↺

Update the CodePen example to enable the following for each row of numbers (1–3):

- ROW 1 Justify-content:center aligns the items in the centre of the container
- ROW 2 Justify-content:space-around aligns the items with equal space either side of each item
- ROW 3 Justify-content:space-between aligns the items with equal space between the items, but not at the ends of the container
- ROW 4 Justify-content:flex-start places the items at the start of the container. Using flex-end will do the opposite.

☒ I have successfully completed this task!



How did you find the task? Here is the example code that we used:

Example 6.16 Task: Flexbox justification

Previous attempts ▼



[Skip to main content](#)

Building on the basics of flexbox, let's now explore additional layout options, such as alignment, spacing and advanced properties, to create more dynamic and precise designs.



Thoughts on this page?

